LIMKOKWING UNIVERSITY OF CREATIVE TECHNOLOGY
SIERRA LEONE

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

MODULE ID - MODULE NAME

Title : Individual Assignment – Mini Library Management System
Issue Date : Week 2
Due Date : Week 4
Lecturer/Examiner :
Name of Student/s : Osman Aamir Kamara
Student ID No. : 905005240
Class : BIT 1102F
Year/Semester : 2/1

Academic Honesty Policy Statement

I/We, hereby attest those contents of this attachment are my own work. Referenced works, articles, art, programs, papers or parts thereof are acknowledged at the end of this paper. This includes data excerpted from CD-ROMs, the Internet, other private networks, and other people's disk of the computer system.

Student's Signature: Date: 26th/10/2025

LECTURER'S COMMMENTS/GRADE:

for office use only upon receive

Remark

DATE:
TIME:

RECEIVER'S NAME:

Mini Library Management System (Python) – Design Rationale & UML Documentation

1. Introduction

This document explains the design rationale and structure of the Mini Library Management System developed in Python.
The system was built using dictionaries, lists, and tuples to manage data efficiently and supports CRUD operations as well as book borrowing and returning.

2. Data Structure Design Rationale

The system relies on three main Python data structures dictionaries, lists, and tuples each selected for its specific advantages in representing different parts of the library system.

a) Dictionary (for Books):
A dictionary is used to store books, where each key represents a unique ISBN, and its value stores book details such as title, author, genre, and total copies.
This ensures quick lookups, updates, and deletions. Example:
books = {'001': {'title': 'Python Basics', 'author': 'John Doe', 'genre': 'Education', 'total_copies': 3}}

b) List (for Members):
A list of dictionaries is used to manage library members. Each dictionary stores member information such as ID, name, email,
and a list of borrowed books. Lists allow easy iteration, addition, and dynamic resizing as new members join. Example:
members = [{'member_id': 'M001', 'name': 'Alice', 'email': 'alice@email.com', 'borrowed_books': []}]

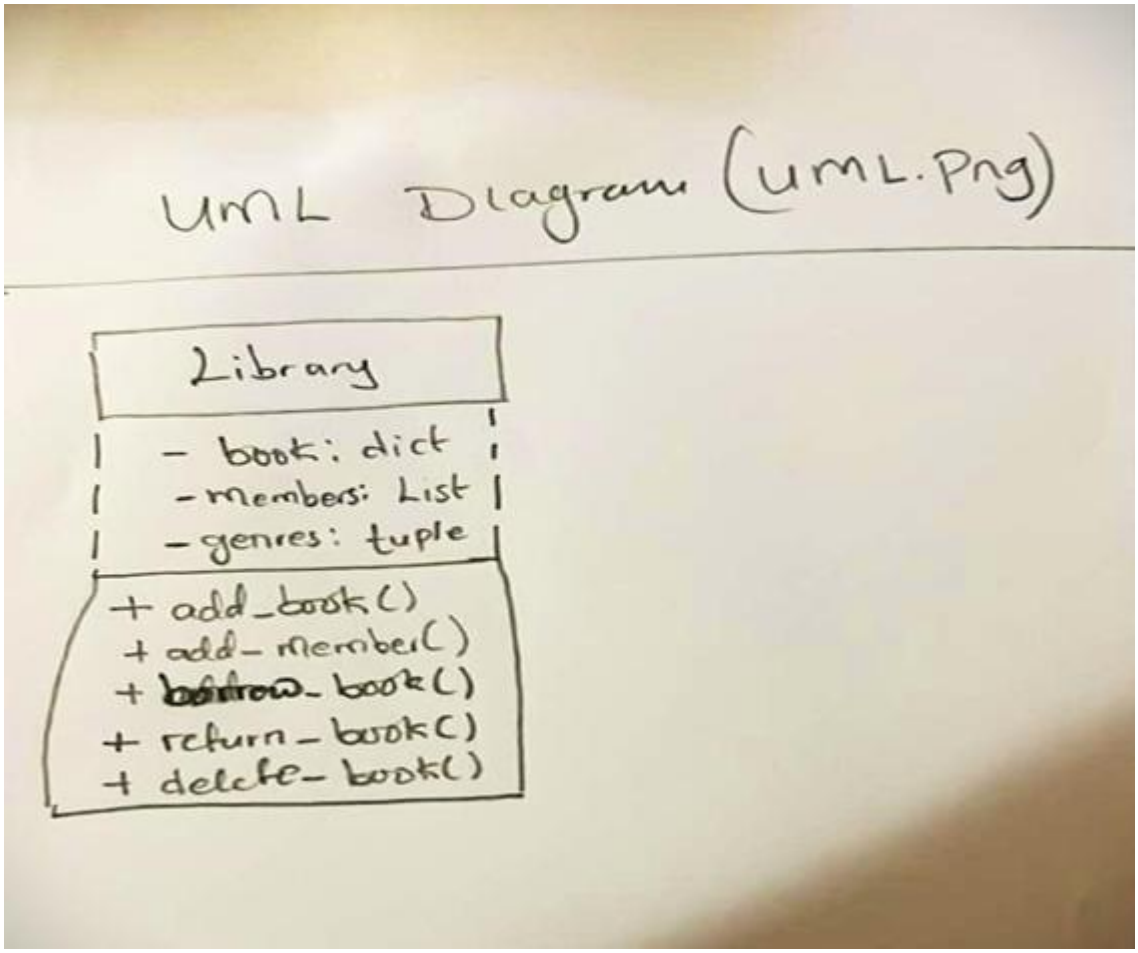
c) Tuple (for Genres):
A tuple stores a fixed list of valid book genres such as Fiction, Non-Fiction, and Sci-Fi. Tuples were chosen because they are immutable,
ensuring that genre values remain constant throughout program execution.

3. Core Functions

The system provides several key functions that perform Create, Read, Update, and Delete (CRUD) operations, as well as borrowing and returning books:
- add_book() / add_member(): Add new books and members if identifiers are unique.
- update_book() / update_member(): Modify existing records.
- delete_book() / delete_member(): Remove entries only if conditions are satisfied (no borrowed books).
- borrow_book() / return_book(): Manage lending and returning of books with checks for limits and availability.

4. UML Diagram (Textual Description)

The UML structure represents three core entities — Book, Member, and Library System.



5. Testing

To ensure the program works as intended, at least five test cases were created using Python's assert statement.
These tests verify correct handling of edge cases such as adding duplicate books, borrowing books when no copies are available, and ensuring limits on borrowed books.
Examples of test cases include:
- Adding a valid book entry.
- Rejecting duplicate ISBNs.
- Borrowing a book successfully.
- Preventing borrowing when no copies remain.
- Returning a book correctly.

6. Demo Script

A demo script nameddemo.py is included in the system to demonstrate its functionality. It performs the following tasks:
1. Adds new books and members.
2. Displays search results.
3. Demonstrates borrowing and returning operations.
4. Deletes members and books to show CRUD functionality.
This script provides a clear end-to-end demonstration of how the Mini Library Management

System operates.

7. Conclusion

The Mini Library Management System demonstrates practical use of Python's built-in data structures to implement a robust, efficient,
and easily extendable information management system. The project successfully integrates CRUD, borrowing, and returning functionality
within a simple and intuitive interface, providing a strong foundation for future upgrades such as GUI or database integration.

```python
# Mini Library Management System (Python)

# ------------------------------
# 1. Data Structures
# ------------------------------

# Dictionary for books: {ISBN: {title, author, genre, total_copies}}
books = {}

# List of members: [{member_id, name, email, borrowed_books}]
members = []

# Tuple for valid genres
genres = ("Fiction", "Non-Fiction", "Sci-Fi", "Biography", "Education")

# ------------------------------
# 2. Core Functions (CRUD + Borrow/Return)
# ------------------------------

def add_book(isbn, title, author, genre, total_copies):
    if isbn in books:
        return "Book already exists."
    if genre not in genres:
        return "Invalid genre."
    books[isbn] = {"title": title, "author": author, "genre": genre,
"total_copies": total_copies}
    return "Book added successfully."

def add_member(member_id, name, email):
    for m in members:
        if m['member_id'] == member_id:
            return "Member ID already exists."
    members.append({"member_id": member_id, "name": name, "email": email,
"borrowed_books": []})
    return "Member added successfully."
```

```
def search_books(keyword):
    result = []
    for isbn, info in books.items():
        if keyword.lower() in info['title'].lower() or keyword.lower() in
info['author'].lower():
            result.append((isbn, info))
    return result

def update_book(isbn, title=None, author=None, genre=None, total_copies=None):
    if isbn not in books:
        return "Book not found."
    if genre and genre not in genres:
        return "Invalid genre."
    if title: books[isbn]['title'] = title
    if author: books[isbn]['author'] = author
    if genre: books[isbn]['genre'] = genre
    if total_copies: books[isbn]['total_copies'] = total_copies
    return "Book updated successfully."

def update_member(member_id, name=None, email=None):
    for m in members:
        if m['member_id'] == member_id:
            if name: m['name'] = name
            if email: m['email'] = email
            return "Member updated successfully."
    return "Member not found."

def delete_book(isbn):
    if isbn not in books:
        return "Book not found."
    for m in members:
        if isbn in m['borrowed_books']:
            return "Cannot delete. Book currently borrowed."
    del books[isbn]
    return "Book deleted successfully."

def delete_member(member_id):
    for m in members:
        if m['member_id'] == member_id:
            if len(m['borrowed_books']) > 0:
                return "Cannot delete member with borrowed books."
            members.remove(m)
            return "Member deleted successfully."
    return "Member not found."

def borrow_book(member_id, isbn):
    for m in members:
        if m['member_id'] == member_id:
```

```
            if len(m['borrowed_books']) >= 3:
                return "Cannot borrow more than 3 books."
            if isbn not in books:
                return "Book not found."
            if books[isbn]['total_copies'] <= 0:
                return "No copies available."
            m['borrowed_books'].append(isbn)
            books[isbn]['total_copies'] -= 1
            return f"{m['name']} borrowed {books[isbn]['title']}."
    return "Member not found."

def return_book(member_id, isbn):
    for m in members:
        if m['member_id'] == member_id:
            if isbn not in m['borrowed_books']:
                return "Book not borrowed by this member."
            m['borrowed_books'].remove(isbn)
            books[isbn]['total_copies'] += 1
            return f"{m['name']} returned {books[isbn]['title']}."
    return "Member not found."

# ------------------------------
# 3. Testing using assert
# ------------------------------

def run_tests():
    assert add_book("001", "Python Basics", "Osman Aamir", "Education", 3) ==
"Book added successfully."
    assert add_book("001", "Python Basics", "Osman Aamir", "Education", 3) ==
"Book already exists."
    assert add_member("M001", "Aamir", "aamir@email.com") == "Member added
successfully."
    assert borrow_book("M001", "001") == "Aamir borrowed Python Basics."
    assert borrow_book("M001", "001") != "No copies available."  # At least
one copy still available
    assert return_book("M001", "001") == "Aamir returned Python Basics."
    print("All tests passed successfully!")

# ------------------------------
# 4. Demo Script
# ------------------------------

def demo():
    print(add_book("002", "Advanced Python", "Gladys Boima", "Education", 2))
    print(add_member("M002", "Con", "con@email.com"))
    print(search_books("Python"))
    print(borrow_book("M002", "002"))
    print(return_book("M002", "002"))
```

```
    print(delete_member("M002"))
    print(delete_book("002"))

# Run demo and tests if executed directly
if __name__ == "__main__":
    run_tests()
    demo()
```

PS C:\Users\Osman Aamir Kamara> & "C:/Users/Osman Aamir Kamara/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Osman Aamir Kamara/Desktop/# Mini Library Management System (Python.py"

All tests passed successfully!

Member added successfully.

[('001', {'title': 'Python Basics', 'author': 'Osman Aamir', 'genre': 'Education', 'total_copies': 2}), ('002', {'title': 'Advanced Python', 'author': 'Gladys Boima', 'genre': 'Education', 'total_copies': 2})]

Con borrowed Advanced Python.

Con returned Advanced Python.

Member deleted successfully.

Book deleted successfully.

PS C:\Users\Osman Aamir Kamara>